TECHLEARN ACADEMY

Guia de estudo do programa

PIM IV · Código web e API, banco SQL, entregas desktop e Android

Para quatro estudantes de ADS. Este guia usa o projeto local e as entregas disponíveis em 27/10/2026.

O foco é entender o caminho de uma ação, explicar as decisões do código e reconhecer os limites do que foi possível verificar.

Como usar	O que fazer
Primeira leitura	Leia as seções 1 e 2 para enxergar o sistema inteiro.
Estudo do código	Siga um exemplo completo: tela → JavaScript → controller → serviço → SQL → resposta.
Banca	Use o roteiro da seção 9 e explique cada passo com suas próprias palavras.

Regra deste documento: “confirmado no código” significa que o arquivo foi lido; “informado na documentação” é uma declaração da entrega; “não verificado” exige fonte ou execução adicional.

Sumário

1. VISÃO GERAL 3

2. MAPA DO PROJETO 5

3. BACK-END EM C# 6

4. FRONT-END WEB 9

5. API REST E INTEGRAÇÕES 11

6. LOGIN E SEGURANÇA 13

7. BANCO DE DADOS 14

8. DESKTOP E MOBILE: NÃO MISTURAR VERSÕES 17

9. ROTEIRO PARA A BANCA 18

Convenção de caminhos

Os caminhos do código citados a seguir são relativos a `C:\Users\Lfgr1\Downloads\TechLearnAcademy-PIMIV-NOVO\TechLearnAcademy-PIMIV`. As entregas estão em `C:\Users\Lfgr1\Downloads\entregas`. O número após dois pontos indica a linha do arquivo nesta cópia local.

Limite da verificação: o código e os pacotes locais foram examinados. Não foram executados o site publicado, o banco Azure, o executável desktop nem o APK nesta análise. A documentação pode descrever testes anteriores, que não substituem uma nova execução aqui.

1. VISÃO GERAL

A TechLearn Academy oferece cursos de programação: o aluno se matricula, estuda módulos, responde avaliações e, se aprovado, solicita certificado. Mentores atendem dúvidas; gestores mantêm cursos, avaliam solicitações e acompanham resultados; administradores cuidam das contas e de chamados encaminhados pela gestão.

Perfil	O que faz no código atual
Aluno	Cadastro público, curso, matrícula, material, avaliação, resultado, ajuda e certificado.
Mentor	Atende chamados de ajuda e registra orientações aos alunos atendidos.
Gestor	Gerencia cursos, conteúdo, avaliações, alunos/mentores, relatórios, certificados e chamados.
Administrador	Administra usuários e atende chamados da gestão; não troca nem recupera senha pelo site.

Confirmado no código: `TechLearnAcademy.Web/wwwroot/js/api.js:102-117`.

Confirmado no código: `TechLearnAcademy.Api/Controllers/GestaoController.cs:11-58`.

Versões encontradas

Componente	O que foi examinado	Estado da afirmação
Web + API	Fonte local ASP.NET Core/C#/HTML/CSS/JS; pacote Azure de 21/10. 40 arquivos web locais iguais aos publicados; DLL Release igual à do pacote.	Confirmado nos arquivos locais. Estado ao vivo do Azure não verificado.
Desktop no ZIP de código	Projeto C# Windows Forms com navegador WebView2 em TelaPrincipal.cs.	Confirmado no código desta pasta/ZIP. É versão anterior.
Desktop entregue	EXE 1.0.2 de 25/10; metadados dizem TechLearn Academy - Administração e o pacote contém referência a Windows Forms.	Metadados confirmados; telas nativas e funções descritas no LEIA-ME. Fonte nativa ausente.
Mobile no ZIP de código	Projeto .NET MAUI com WebView em MainPage.xaml.cs.	Confirmado no código desta pasta/ZIP. É versão anterior.
Android entregue	APK de 25/10 com Manifest, dois DEX e nomes de Activities Java; README descreve telas nativas.	Estrutura do APK confirmada; implementação detalhada e uso visual não verificados por falta dos fontes/aparelho.

Informado na documentação e amostrado por hash: `Documentacao/sincronizacao-azure-20261021.md:3-9`.

O arquivo ZIP `TechLearnAcademy-PIMIV-NOVO (5).zip` (21/10) contém o desktop com WebView2 e o mobile MAUI; não contém os fontes nativos descritos nas entregas de 25/10.

Tecnologias reais

Tecnologia	Onde aparece	Função
C# / .NET 10 / ASP.NET Core	API e projeto Web	Receber HTTP, aplicar regras, responder JSON e servir páginas.
HTML / CSS / JavaScript	TechLearnAcademy.Web/wwwroot	Estrutura, aparência e interação no navegador.
SQL Server / Azure SQL	Banco/*.sql e ConexaoBanco.cs	Guardar usuários, cursos, notas, sessões, chamados e certificados.
Microsoft.Data.SqlClient	TechLearnAcademy.Api.csproj	Abrir conexão e executar SQL com parâmetros.
Azure App Service	publicacao/azure/site	Hospedar o pacote da API e do site.
Azure Communication Email	UsuariosServico.cs	Enviar link real de recuperação no Azure, se configurado.
VLibras	wwwroot/js/config.js	Carregar botão externo de tradução para Libras.
Windows Forms / WebView2	Fonte desktop antigo	Janela Windows com navegador interno.
Java Android nativo	APK entregue + LEIA-ME	Cliente móvel descrito como telas Activity; fonte não disponível.

Confirmado no código: `TechLearnAcademy.Api/TechLearnAcademy.Api.csproj:1-13`.

Confirmado no código: `TechLearnAcademy.Web/wwwroot/js/config.js:1-8`.

O caminho de uma ação



Exemplo: clicar em **Entrar** envia JSON para `POST /api/usuarios/login`. O controller chama o serviço; o serviço verifica o usuário no SQL Server; a API devolve dados e cookie; o JavaScript mostra a mensagem e abre o painel do perfil. O banco é acessado pela API, não diretamente pela página.

2. MAPA DO PROJETO

Caminho real (a partir da raiz acima)	O que ler primeiro
TechLearnAcademy.Api/Program.cs	Inicialização, serviços, autenticação, erros e entrega do site.
TechLearnAcademy.Api/Controllers/	Rotas HTTP, verbos e perfis permitidos.
TechLearnAcademy.Api/Servicos/	Regras de negócio: login, cursos, provas, gestão e sessão.
TechLearnAcademy.Api/Modelos/	Nove classes de domínio e Pedidos.cs com dados de entrada.
TechLearnAcademy.Api/Banco/	ConexaoBanco.cs: conexão, parâmetros, consulta e gravação.
TechLearnAcademy.Web/wwwroot/	Páginas .html, estilo.css e arquivos JavaScript.
Banco/	Instalação e atualizações SQL; ler 01 e depois 04-07 ou o instalador consolidado.
Documentacao/	Guias e histórico; conferir sempre as afirmações antigas contra o código.
Diagramas/	Modelos Astah; ajudam na explicação, mas o código decide o comportamento real.
TechLearnAcademy.Tests/ e Testes/	Fontes e registros de testes anteriores; não executados nesta análise.
publicacao/azure/site e site.zip	Artefatos de publicação, não fonte para editar.
TechLearnAcademy.Desktop/ e .Mobile/	Fontes antigos WebView2 e MAUI; não os confundir com as entregas nativas.

Código, histórico e saídas

• **Código atual web/API:** edite os projetos Api e Web; o `.csproj` da API inclui o `wwwroot` da Web na publicação.

• **Scripts de evolução:** `01-criar-banco.sql` define a base; `04`, `05` e `07` representam mudanças posteriores. `06` carrega cursos e PDFs. `Instalar-banco-do-site.sql` reúne instalação e atualizações.

• **Gerados e publicados:** `bin/`, `obj/`, `publicacao/azure/site/`, ZIPs e BACPACs são saídas/cópias. Não começar a leitura por eles.

• **Entregas fora da solução:** `entregas/Desktop` contém EXE, README e endereço do servidor; `entregas/Android` contém APK e README. Não há nessa pasta os fontes nativos listados pelos READMEs.

Ordem prática para localizar uma função: ache o botão no HTML → descubra a função no JS → copie a rota `/api/...` → busque a rota em Controllers → leia o método em Servicos → encontre a tabela no SQL.

3. BACK-END EM C#

Quatro peças e uma requisição

Program.cs configura o aplicativo: registra as classes de serviço, autenticação por sessão, CORS, tratamento de erro, páginas estáticas e controllers. **Controller** traduz rota HTTP em chamada C#. **Serviço** toma decisões, valida regras e consulta/grava. **ConexaoBanco** executa comandos SQL. **Modelos** representam objetos do domínio ou dados recebidos.

Trecho real: `TechLearnAcademy.Api/Program.cs:14-20` (linhas selecionadas).

```
14  builder.Services.AddScoped<ConexaoBanco>();
15  builder.Services.AddScoped<UsuariosServico>();
16  builder.Services.AddScoped<CursosServico>();
17  builder.Services.AddScoped<AvaliacoesServico>();
18  builder.Services.AddScoped<GestaoServico>();
19  builder.Services.AddAuthentication("Sessao").AddScheme<AuthenticationSchemeOptions,
    SessaoHandler>("Sessao", null);
20  builder.Services.AddAuthorization();
```

L14. Registra a classe de acesso ao SQL para cada solicitação.

L15. Registra as regras de usuários.

L16. Registra as regras de cursos e matrículas.

L17. Registra a correção de avaliações.

L18. Registra a gestão: chamados, relatórios, certificados e usuários.

L19. Seleciona a autenticação personalizada chamada Sessao.

L20. Habilita autorização por perfil nos controllers.

O mesmo padrão de controller fino aparece em `CursosController.cs`, `AvaliacoesController.cs` e `GestaoController.cs`.

Trecho real: `TechLearnAcademy.Api/Controllers/UsuariosController.cs:19-20` (linhas selecionadas).

```
19      [HttpPost("login")]
20      public async Task<IActionResult> Login(DadosLogin dados) => Ok(await
    servico.Login(dados, Response, Request.IsHttps));
```

L19. Associa POST e o trecho login à rota-base api/usuarios da linha 8.

L20. ASP.NET transforma o JSON em DadosLogin, chama o serviço e devolve HTTP 200 com o resultado.

Como o SQL recebe valores

Trecho real: `TechLearnAcademy.Api/Banco/ConexaoBanco.cs:20-28` (linhas selecionadas).

```
20      public static SqlCommand Comando(SqlConnection banco, string sql, object? dados = null,
 SqlTransaction? transacao = null)
22          var comando = new SqlCommand(sql, banco, transacao);
25              foreach (var campo in dados.GetType().GetProperties())
26                  comando.Parameters.AddWithValue("@" + campo.Name, campo.GetValue(dados) ??
DBNull.Value);
28          return comando;
```

L20. O método recebe conexão, texto SQL, dados e uma transação opcional.

L22. Cria o comando SQL ligado à conexão recebida.

L25. Percorre as propriedades do objeto anônimo enviado pelo serviço.

L26. Transforma cada propriedade em parâmetro @nome; NULL vira DBNull.Value.

L28. Entrega o comando pronto para Consultar, Gravar ou Numero.

Exemplo recorrente: `UsuariosServico.cs:62` usa `WHERE Email=@email` com `new { email }`. O valor digitado não vira texto da instrução SQL. As buscas de curso, aluno e chamado seguem a mesma ideia.

Login: da regra à sessão

Trecho real: `TechLearnAcademy.Api/Servicos/UsuariosServico.cs:62-75` (linhas selecionadas).

```
62          var lista = await ConexaoBanco.Consultar(banco, "SELECT Id AS id,Nome AS nome,Email
 AS email,Perfil AS perfil,SenhaHash AS senhaHash FROM Usuarios WITH(UPDLOCK,HOLDLOCK)
WHERE Email=@email AND Ativo=1", new { email }, transacao);
65          Regras.Exigir(usuario != null && usuario.ConferirSenha(dados.Senha), "E-mail ou
senha inválidos.", 401);
66          string token = Regras.NovoToken();
68          await ConexaoBanco.Gravar(banco, "INSERT INTO Sessoes(UsuarioId,TokenHash,Validade)
 VALUES(@usuarioId,@hash,@validade)", new { usuarioId = usuario!.Id, hash =
Regras.HashToken(token), validade }, transacao);
69          await transacao.CommitAsync();
70          resposta.Cookies.Append("TechLearnSessao", token, new CookieOptions
72              HttpOnly = true, Secure = https, SameSite = SameSiteMode.Strict,
75          return new { mensagem = "Login realizado com sucesso.", id = usuario!.Id, nome =
usuario.Nome, email = usuario.Email, perfil = usuario.Perfil, token };
```

L62. Busca uma conta ativa pelo e-mail parametrizado.

L65. Compara a senha digitada com o hash salvo; se errar, lança 401.

L66. Cria um token aleatório novo.

L68. Salva somente o hash do token na tabela Sessoes, com validade.

L69. Confirma a transação do banco.

L70. Envia um cookie chamado TechLearnSessao ao navegador.

L72. HttpOnly impede leitura pelo JS; Secure depende de HTTPS; SameSite é Strict.

L75. Devolve dados do usuário e o token no JSON para clientes que usam Bearer.

No pedido seguinte, `SessaoHandler.cs:14-37` lê `Authorization: Bearer` ou cookie, calcula o hash e consulta `Sessoes` + `Usuarios`. Só uma sessão válida de usuário ativo cria a identidade com perfil.

Erro apresentado no site

Trecho real: `TechLearnAcademy.Api/Program.cs:93-110` (linhas selecionadas).

```
93      catch (ErroAplicacao erro)
95          contexto.Response.StatusCode = erro.Status;
96          await contexto.Response.WriteAsJsonAsync(new { mensagem = erro.Message });
103         string mensagem = "Não foi possível acessar o SQL Server. Verifique a conexão e a
criação do banco da nova solução.";
104         if (erro.Number == 2601 || erro.Number == 2627) { status = 409; mensagem = "Já
existe um registro com esses dados."; }
109         contexto.Response.StatusCode = status;
110         await contexto.Response.WriteAsJsonAsync(new { mensagem });
```

L93. Captura um ErroAplicacao criado por uma validação.

L95. Aplica o status HTTP definido pelo erro.

L96. Envia JSON com a mensagem legível.

L103. Escolhe 503 quando o SQL não pode ser usado.

L104. Converte duplicidade SQL em conflito 409 e mensagem neutra.

L109. Aplica o status escolhido.

L110. Devolve o erro como { mensagem }, que o JS mostra na página.

Para erro inesperado, `Program.cs:112-117` registra apenas o tipo/caminho e devolve 500 genérico. Isso evita mostrar detalhes internos ao usuário.

As nove classes centrais

Classe	Dados e operação relevante	Relação
Usuario	Id, Nome, Email, Perfil, Ativo, SenhaHash; AtualizarDados, DefinirSenha, ConferirSenha	Conta de Aluno/Mentor/Gestor/Administrador.
Curso	Nome, Descricao, CargaHoraria; construtor valida	Um curso possui módulos no banco.
Modulo	Titulo, Ordem; ParaCadastro	CursoId está na tabela Modulos.
Material	Titulo, Tipo, Url; valida link/PDF	ModuloId está na tabela Materiais.
Matricula	AlunoId, CursoId, TentativaCurso; Reiniciar	Liga aluno e curso.
Avaliacao	ModuloId, Titulo; construtor	Avaliação pertence a um módulo.
Questao	AvaliacaoId, Enunciado, Alternativas; valida 2 a 5	Questão pertence à avaliação.
Resultado	Acertos, Total, Percentual, Nota; Aprovado >= 60%	Calcula o resultado usado na correção.
OrientacaoMentor	MentorId, AlunoId, Texto	Registra orientação do mentor ao aluno.

Confirmado no código: `TechLearnAcademy.Api/Modelos/Usuario.cs:6-50`.

Confirmado no código: `TechLearnAcademy.Api/Modelos/Resultado.cs:6-21`.

Onde há POO: cada objeto reúne dados e regras de validação/cálculo. Por exemplo, `Matricula.Reiniciar` muda a tentativa e zera o progresso, e `Resultado` calcula a nota. Os serviços também são classes que recebem `ConexaoBanco` pelo construtor. Não há necessidade de inventar herança entre Aluno e Mentor: o código usa `Usuario.Perfil`.

Distinção importante: `Modelos/Pedidos.cs` define classes como `DadosLogin`, `DadosCurso` e `DadosProva`. Elas transportam JSON recebido; não são as nove entidades de negócio.

Confirmado no código: `TechLearnAcademy.Api/Modelos/Pedidos.cs:1-168`.

4. FRONT-END WEB

Quem faz o quê

Parte	Exemplo real
HTML	index.html:34-49 cria o formulário de login, os campos, o botão e o espaço da mensagem.
CSS	css/estilo.css:23-27 diferencia avisos de erro e sucesso; linhas 83-94 ajustam o layout estreito.
JavaScript	js/publico.js reage ao login; js/api.js faz fetch e atualiza o DOM.

Um exemplo completo: entrar

Trecho real: `TechLearnAcademy.Web/wwwroot/js/publico.js:3-12` (linhas selecionadas).

```
3   if (pagina === "login") {
4       enviarFormulario("form-login", async function () {
5           const email = textoCampo("email");
6           verificarEmail(email);
7           const usuario = await requisicao("/api/usuarios/login", "POST", { email: email,
    senha: valor("senha") });
8           const destino = destinoPerfil(usuario.perfil);
10          aviso("Login realizado com sucesso.", "sucesso");
12          setTimeout(function () { location.href = destino; }, 600);
```

L3. Esta parte do JS roda na página de login.

L4. Associa uma tarefa ao envio do formulário.

L5. Lê o e-mail do campo HTML.

L6. Confere o formato antes de enviar.

L7. Envia e-mail e senha para a API usando POST/JSON.

L8. Escolhe o painel a partir do perfil que a API devolveu.

L10. Mostra o aviso de sucesso dentro do site.

L12. Após breve espera, abre a página do perfil; o token não é salvo pelo JS.

Trecho real: `TechLearnAcademy.Web/wwwroot/js/api.js:23-46` (linhas selecionadas).

```
23  async function requisicao(caminho, metodo = "GET", dados) {
24      const opcoes = { method: metodo, credentials: "include", cache: "no-store" };
28              opcoes.headers = { "Content-Type": "application/json" };
29              opcoes.body = JSON.stringify(dados);
33      try { resposta = await fetch(URL_API + caminho, opcoes); }
34      catch { throw new Error("Não foi possível conectar ao servidor. Verifique se a API está
     ligada."); }
41      if (!resposta.ok) {
42          const erro = new Error(retorno.mensagem || "Não foi possível concluir a
    solicitação.");
44          throw erro;
46      return retorno;
```

L23. Função comum para qualquer rota da API.

L24. Define método, envia cookie e pede resposta nova.

L28. Indica que o corpo é JSON.

L29. Transforma objeto JS em texto JSON.

L33. fetch envia a solicitação.

L34. Falha de rede vira mensagem compreensível.

L41. Detecta status HTTP de erro.

L42. Usa a mensagem enviada pelo servidor, se existir.

L44. Lança o erro para o formulário mostrar.

L46. Entrega o JSON quando deu certo.

O helper `enviarFormulario` impede recarregar a página, usa a validação nativa do formulário, desativa o botão durante o envio e mostra erro sem depender do Visual Studio (`js/api.js:160-174`). `aviso` escreve com `textContent` no elemento `#mensagem` e foca avisos de erro (`js/api.js:11-20`).

Perfil e atualização da tela

Após entrar, `js/api.js:127-157` chama `GET /api/usuarios/perfil`, monta o menu, exibe nome/perfil e bloqueia visualmente uma página inadequada. Isso organiza a navegação. A proteção real está no servidor, nos `[Authorize(Roles=...)]` dos controllers e na verificação da sessão; esconder botão não protege rota.

O servidor anexa uma versão aos JS/CSS do HTML (`Program.cs:136-159`); o navegador consulta `/versao-site` a cada cinco segundos (`js/api.js:281-312`). Isso ajuda a perceber arquivos novos sem exigir que o aluno adivinhe quando atualizar a página.

`js/config.js:4-8` carrega o plugin externo VLibras. A interface web usa JavaScript próprio, mas esta integração externa existe e deve ser citada na apresentação.

5. API REST E INTEGRAÇÕES

Uma API REST é um conjunto de endereços HTTP que recebem pedidos e devolvem dados. Nesta aplicação, cada rota começa em `/api`; as páginas usam `fetch` e recebem JSON. GET lê; POST cria ou executa uma ação; PUT altera; DELETE exclui logicamente ou remove conforme a função.

Método e rota	Quem	Entrada → saída	Classe
POST /api/usuarios/cadastro	Público	DadosCadastro → 201 + mensagem	UsuariosController
POST /api/usuarios/login	Público	DadosLogin → perfil, cookie e token	UsuariosController
GET /api/usuarios/perfil	Logado	sem corpo → perfil atual	UsuariosController
POST /api/usuarios/logout	Logado	sem corpo → mensagem	UsuariosController
POST /api/usuarios/recuperar-senha	Público	e-mail → mensagem	UsuariosController
GET /api/cursos	Aluno/Gestor	busca, pagina → itens e total	CursosController
POST /api/cursos	Gestor	DadosCurso → 201 + id	CursosController
PUT /api/cursos/{id}	Gestor	DadosCurso → mensagem	CursosController
DELETE /api/cursos/{id}	Gestor	id → mensagem	CursosController
POST /api/matriculas	Aluno	cursoId → 201 + mensagem	CursosController
POST /api/cursos/{id}/reiniciar	Aluno	id → mensagem	CursosController
POST /api/avaliacoes/{id}/respostas	Aluno	respostas → nota/aprovação	AvaliacoesController
GET /api/resultados	Aluno	sem corpo → histórico	AvaliacoesController
POST /api/chamados	Aluno/Mentor/Gestor	DadosChamado → 201 + id	GestaoController
POST /api/chamados/{id}/assumir	Mentor/Gestor/Admin	id → mensagem	GestaoController
POST /api/certificados	Aluno	cursoId → mensagem	GestaoController
POST /api/certificados/{id}/decisao	Gestor	aprovar/motivo → mensagem	GestaoController
GET /api/relatorios/cursos/{id}/alunos	Gestor	id → alunos e progresso	GestaoController
PUT /api/administracao/usuarios/{id}	Admin/Gestor*	DadosUsuario → mensagem	GestaoController

*O serviço restringe o que um gestor pode editar; a permissão do atributo da rota não é a única regra. Rotas e perfis: `Controllers/UsuariosController.cs:8-60`, `CursosController.cs:8-67`, `AvaliacoesController.cs:8-39` e `GestaoController.cs:8-58`.

JSON sem dados reais

Exemplo **ilustrativo** da estrutura usada pelo código: o gestor envia `POST /api/cursos` com `{"nome":"Lógica inicial","descricao":"Introdução","cargaHoraria":20}`. O controller responde com HTTP 201 e algo como `{"id":123,"mensagem":"Curso cadastrado."}`. O número 123 é fictício; a rota e os nomes dos campos vêm de `CursosController.cs:17-18` e `Modelos/Pedidos.cs:30-36`.

Se a API não responder

Status	O que significa aqui	Exemplo de reação
Sem conexão	O fetch não alcançou o servidor.	Mostra “Não foi possível conectar ao servidor”.
400	Dados inválidos.	Corrigir campos/alternativas.
401	Sessão ausente/vencida ou login inválido.	Entrar novamente.
403	Perfil sem permissão ou ação proibida.	Usar o perfil correto.
404	Registro/rota não encontrado.	Conferir link ou ID.
409	Conflito: e-mail repetido, prova já respondida, curso com limite de módulos etc.	Ler a mensagem e corrigir a ação.
500	Erro inesperado do servidor.	Verificar log, sem expor detalhes ao usuário.
503	SQL ou envio de e-mail indisponível/mal configurado.	Conferir serviço e configuração.

Confirmado no código: `TechLearnAcademy.Web/wwwroot/js/api.js:32-46`.

Confirmado no código: `TechLearnAcademy.Api/Program.cs:74-117`.

Web chama a API na mesma origem. O desktop antigo e o mobile antigo abrem o próprio site em WebView. As entregas nativas declaram chamar a mesma API por HTTPS/JSON; o pacote desktop e o APK não contêm uma conexão SQL a ser configurada pelo usuário. O funcionamento detalhado das versões entregues não foi confirmado sem seus fontes.

6. LOGIN E SEGURANÇA

Termo	Aplicação real
Hash da senha	PBKDF2 com SHA-256, sal aleatório e 600.000 iterações; não salva a senha em texto. Regras.cs:48-75.
Sessão	Token aleatório de 32 bytes vira texto hexadecimal; o SQL guarda SHA-256 do token em Sessoes. UsuariosServico.cs:66-75.
Cookie	TechLearnSessao é HttpOnly e SameSite=Strict; Secure quando HTTPS. O navegador o envia automaticamente.
Bearer	Clientes podem enviar o token em Authorization: Bearer; SessaoHandler aceita essa alternativa.
JWT	Não é usado. O token é opaco: o servidor consulta o hash e a validade no SQL a cada pedido.
HTTPS	Na produção Program.cs habilita redirecionamento e HSTS. Os aplicativos antigos exigem endereço HTTPS.
Autorização	[Authorize] e Roles nos controllers + regras no serviço; o perfil vem da sessão consultada.
CORS	Só adiciona origens configuradas em OrigensPermitidas; não é substituto da autorização.
SQL parametrizado	ConexaoBanco.Comando cria @parâmetros a partir dos dados; evita concatenar valores do usuário.

Confirmado no código: `TechLearnAcademy.Api/Servicos/Regras.cs:48-81`.

Confirmado no código: `TechLearnAcademy.Api/Servicos/SessaoHandler.cs:14-49`.

Senha e recuperação

A regra atual é **mínimo de 8 caracteres, sem espaço, com maiúscula, número e símbolo** (`Regras.cs:48-56`). O LEIA-ME antigo fala em 15 caracteres e espaços permitidos; para a banca, use a regra do código atual. No site, aluno, mentor e gestor podem trocar ou recuperar; o administrador não. A recuperação gera link de uso único por 30 minutos e, depois de usada, revoga sessões antigas.

No Azure, `UsuariosServico.cs:149-180` escolhe **Azure Communication Services Email** quando `Email:ConexaoAzure` está configurada; caso contrário, tenta SMTP configurado. Não expor valores de configuração. A documentação registra envio e redefinição confirmados em outubro, mas não fiz novo teste de e-mail nesta análise.

Informado na documentação: `Documentacao/recuperacao-senha-azure.md:5-37`.

Rotas: `POST /api/usuarios/login`, `logout`, `recuperar-senha`, `redefinir-senha` e `trocar-senha`. Para o administrador, existe comando local `--redefinir-admin` no `Program.cs:55-71`, sem rota pública.

> **Frase para a banca:** “O login usa um token aleatório guardado como hash no banco. O navegador usa cookie HttpOnly; outros clientes podem usar Bearer. Não é JWT. Em cada pedido o servidor confere sessão, validade, conta ativa e perfil.”

7. BANCO DE DADOS

A API lê a conexão `ConnectionStrings:Banco` da configuração e abre `SqlConnection` em `ConexaoBanco.cs:7-17`. O script `Banco/Instalar-banco-do-site.sql` mostra a estrutura base e as atualizações; `Banco/01` a `07` separam etapas históricas. O BACPAC de 21/10 é cópia de dados, não código.

Tabela(s)	Relação simples
Usuarios, Sessoes, RecuperacoesSenha	Uma conta pode ter sessões e pedidos de recuperação; guardam hash do token.
Cursos → Modulos → Materiais	Um curso contém até quatro módulos ativos; cada módulo tem links/PDFs.
Usuarios + Cursos → Matriculas	A matrícula liga aluno e curso e guarda a tentativa do curso.
Modulos → Avaliacoes → Questoes → Alternativas	Define as provas e a alternativa correta.
TentativasAvaliacao → RespostasAluno	Guarda nota e escolhas da tentativa, preservando histórico.
ProgressoModulos	Marca módulo concluído por aluno.
Chamados → MensagensChamado	Liga solicitante, responsável e conversa.
SolicitacoesCertificado	Liga aluno/curso/gestor e guarda status, código e dados da emissão.
OrientacoesMentor e AuditoriaUsuarios	Orientação do mentor; histórico de alterações de situação/perfil.

Confirmado no código: `Banco/Instalar-banco-do-site.sql:33-202,345-388`.

As classes de domínio não representam todas as tabelas uma a uma. `Usuario` ↔ Usuarios, `Curso` ↔ Cursos, `Matricula` ↔ Matriculas; `Resultado` calcula os campos que vão para TentativasAvaliacao. Chamados e certificados usam classes de dados e serviço, sem criar uma classe de domínio extra só para o diagrama.

Consulta, inserção e transação reais

Trecho real: `TechLearnAcademy.Api/Servicos/CursosServico.cs:15-19` (linhas selecionadas).

```
15          var dados = new { busca = "%" + (busca ?? "").Trim() + "%", todos = todos &&
usuario.Perfil == "Gestor", inicio = (pagina - 1) * 12 };
16          string filtro = " WHERE Excluido=0 AND (Ativo=1 OR @todos=1) AND (Nome LIKE @busca
OR Descricao LIKE @busca)";
17          int total = await ConexaoBanco.Numero(banco, "SELECT COUNT(*) FROM Cursos" +
filtro, dados);
18          var itens = await ConexaoBanco.Consultar(banco, "SELECT Id AS id,Nome AS
nome,Descricao AS descricao,CargaHoraria AS cargaHoraria,Ativo AS ativo FROM Cursos" +
filtro + " ORDER BY Nome,Id OFFSET @inicio ROWS FETCH NEXT 12 ROWS ONLY", dados);
19          return new { itens, pagina, totalPaginas = Math.Max(1, (total + 11) / 12), total };
```

L15. Cria parâmetros para busca, opção do gestor e início da página.

L16. Define o filtro SQL; o texto digitado entra como @busca.

L17. Conta quantos cursos atendem ao filtro.

L18. Obtém só 12 cursos da página pedida.

L19. Devolve lista e dados de paginação ao frontend.

Trecho real: `TechLearnAcademy.Api/Servicos/CursosServico.cs:34-38` (linhas selecionadas).

```
34      public async Task<int> CriarCurso(DadosCurso dados)
36          var curso = new Curso(dados.Nome, dados.Descricao, dados.CargaHoraria,
dados.Ativo);
37          using var banco = await conexao.Abrir();
38          return await ConexaoBanco.Numero(banco, "INSERT INTO
Cursos(Nome,Descricao,CargaHoraria) VALUES(@nome,@descricao,@carga); SELECT
CAST(SCOPE_IDENTITY() AS int)", new { nome = curso.Nome, descricao = curso.Descricao, carga
 = curso.CargaHoraria });
```

L34. Método usado pela rota de criação.

L36. Cria Curso, cujo construtor valida nome, descrição e carga.

L37. Abre conexão com o SQL Server.

L38. Insere com @parâmetros e devolve o ID gerado.

Trecho real: `Banco/Instalar-banco-do-site.sql:236-259` (linhas selecionadas).

```
236     BEGIN TRY
237         BEGIN TRANSACTION;
248         IF EXISTS (SELECT 1 FROM dbo.Matriculas WHERE AlunoId = @AlunoId AND CursoId =
@CursoId AND Ativo = 1)
251         INSERT dbo.Matriculas (AlunoId, CursoId) VALUES (@AlunoId, @CursoId);
254         COMMIT TRANSACTION;
258         IF @@TRANCOUNT > 0 ROLLBACK TRANSACTION;
259         THROW;
```

L236. Inicia um bloco com tratamento de erro.

L237. Abre transação: ou tudo conclui, ou tudo desfaz.

L248. Confere matrícula repetida.

L251. Insere a ligação entre aluno e curso.

L254. Confirma a transação quando deu certo.

L258. Desfaz se ocorreu erro.

L259. Repassa o erro para a API tratar.

`CursosServico.cs:204-209` executa a procedure `sp_RealizarMatricula`. A procedure `sp_RelatorioCurso` está definida em `Instalar-banco-do-site.sql:264-288`, mas a tela atual de relatório consulta por SQL no serviço (`GestaoServico.cs:245-255`); não diga que essa tela executa a procedure sem confirmar. O trigger `tr_AuditoriaUsuarios` roda automaticamente após UPDATE de Usuários e registra mudança de perfil/situação (`Instalar-banco-do-site.sql:292-302`). O trigger `tr_LimiteModulos` impede mais de quatro módulos ativos (`Instalar-banco-do-site.sql:393-399`).

Uma avaliação só por tentativa do curso

Trecho real: `TechLearnAcademy.Api/Servicos/AvaliacoesServico.cs:141-180` (linhas selecionadas).

```
141         int tentativaCurso = await CursosServico.TentativaAtual(banco, usuario.Id, cursoId,
 transacao);
145         Regras.Exigir(dados.TentativaCurso == tentativaCurso, "O curso foi reiniciado após
a abertura desta prova. Volte ao curso e abra a avaliação novamente.", 409);
146         int respondidas = await ConexaoBanco.Numero(banco, "SELECT COUNT(*) FROM
TentativasAvaliacao WHERE AlunoId=@alunoId AND AvaliacaoId=@id AND
TentativaCurso=@tentativaCurso", new { alunoId = usuario.Id, id, tentativaCurso },
transacao);
147         Regras.Exigir(respondidas == 0, "Esta avaliação já foi respondida nesta tentativa
do curso. Consulte seu resultado.", 409);
167         var resultado = new Modelos.Resultado(acertos, questoes.Count);
172         int tentativaId = await ConexaoBanco.Numero(banco, "INSERT INTO TentativasAvaliacao
(AlunoId,AvaliacaoId,TentativaCurso,Acertos,TotalQuestoes,Percentual,Nota,Aprovado)
VALUES(@alunoId,@id,@tentativaCurso,@acertos,@total,@percentual,@nota,@aprovado); SELECT
CAST(SCOPE_IDENTITY() AS int)", new { alunoId = usuario.Id, id, tentativaCurso, acertos,
total, percentual, nota, aprovado }, transacao);
179         await transacao.CommitAsync();
180         return new { id = tentativaId, acertos, total, percentual, nota, aprovado };
```

L141. Lê a tentativa atual do curso com bloqueio de transação.

L145. Rejeita formulário aberto antes de reiniciar o curso.

L146. Conta se a avaliação já foi respondida nesta tentativa.

L147. Se já foi, devolve conflito 409.

L167. Resultado calcula percentual, nota e aprovação.

L172. Grava a tentativa com os valores calculados.

L179. Confirma a transação.

L180. Devolve ID, acertos, total, percentual, nota e aprovado.

A aprovação exige 60% (`Modelos/Resultado.cs:12-20`). Após reprovação, `CursosServico.Reiniciar` aumenta `TentativaCurso`, apaga o progresso atual e preserva notas antigas (`CursosServico.cs:158-169`). O SQL cria índice para essa tentativa e impede e-mail pessoal repetido (`Instalar-banco-do-site.sql:427-442`).

8. DESKTOP E MOBILE: NÃO MISTURAR VERSÕES

Versão	O que o código/pacote permite afirmar	O que ainda falta
Desktop do ZIP	`TechLearnAcademy.Desktop/TelaPrincipal.cs:1-72` usa WinForms + WebView2, valida HTTPS e carrega o site.	É fonte antigo, não explica a interface nativa do EXE entregue.
EXE entregue	Metadados: versão 1.0.2, produto TechLearn Academy - Administração; o pacote contém referência a Windows Forms. README descreve abas nativas de Admin e chamadas HTTPS/JSON.	Fontes ApiCliente.cs, TelaLogin.cs, TelaChamado.cs etc. não foram encontrados. Não mostrar código inventado.
Mobile do ZIP	`TechLearnAcademy.Mobile/MainPage.xaml.cs:1-45` usa MAUI WebView e abre o site.	É fonte antigo, não explica o APK Java entregue.
APK entregue	ZIP do APK contém AndroidManifest.xml, classes.dex/classes2.dex e nomes LoginActivity, ChamadosActivity, CertificadosActivity, HttpURLConnection e PdfDocument. README descreve Java nativo.	Fonte Java não encontrado; sem execução em aparelho aqui. Detalhes de cada tela são informados pela documentação.

A comunicação esperada das entregas é **cliente → HTTPS/JSON → API C# → SQL Server**. A documentação diz que o desktop nativo só permite login de Administrador e que o Android cobre os quatro perfis. Isso não deve ser atribuído aos fontes WebView2/MAUI presentes nesta pasta.

Confirmado estaticamente: EXE 1.0.2 e presença de Windows Forms; APK válido como ZIP Android com Manifest e DEX. **Informado nos READMEs:** controles nativos, fluxos, testes anteriores e chamadas HTTP. **Não verificado aqui:** navegação visual, chamadas em execução, instalação no celular e implementação linha a linha da versão nativa. Para explicar código nativo à banca, peçam os fontes correspondentes às entregas.

Confirmado no código: `TechLearnAcademy.Desktop/TelaPrincipal.cs:1-72`.

Confirmado no código: `TechLearnAcademy.Mobile/MainPage.xaml.cs:1-45`.

Documentação das entregas: `entregas/Desktop/LEIA-ME.md` e `entregas/Android/LEIA-ME.md` (arquivos fora da raiz do projeto).

9. ROTEIRO PARA A BANCA

Demonstração curta, em ordem

Passo	Mostrar	O que dizer
1	Cadastro público de aluno e login	A tela envia JSON; cadastro público cria Aluno; o servidor valida e guarda hash.
2	Catálogo, matrícula e um módulo	CursosController recebe a ação; CursosServico usa a procedure de matrícula.
3	Material PDF/link e avaliação	A API controla acesso; só uma resposta por avaliação na tentativa atual.
4	Resultado e possível reinício	Resultado calcula nota; reprovado só refaz após reiniciar o curso.
5	Certificado elegível	Aluno solicita depois de aprovado; gestor decide; só aprovado gera dados do certificado.
6	Gestor: relatório, conteúdo e chamados	Mostre aluno por curso e a conversa com responsáveis.
7	Admin: usuários e chamados	Admin ativa/desativa conta, cria Mentor/Gestor e atende a gestão.

Demonstrem apenas com contas de teste autorizadas e dados não pessoais. Se não houver aluno aprovado, expliquem o caminho pelo código em vez de fingir uma emissão.

Perguntas prováveis - respostas curtas

Pergunta	Resposta para explicar oralmente
O que é API REST?	São rotas HTTP que recebem dados da tela e devolvem JSON. Ex.: POST /api/matriculas.
Onde há POO?	Usuario, Curso, Matricula e Resultado reúnem dados e regras; serviços organizam operações. Perfil diferencia papéis sem herdar quatro classes.
Como a senha é protegida?	PBKDF2 com sal; SQL guarda hash. O site não consulta senha em texto.
É JWT?	Não. É token aleatório opaco; o servidor guarda o hash e consulta a sessão.
Como evitam SQL injection?	Dados viram parâmetros @nome; não concatenamos valores digitados nas consultas.
E a prova repetida?	O serviço consulta TentativaCurso e recusa segunda resposta; só a reprovação permite reiniciar e criar nova tentativa.
Por que desktop/mobile não usam SQL?	A API centraliza validação e permissão. A entrega declara HTTPS/JSON; os fontes nativos não vieram junto.
Por que há dois códigos desktop/mobile?	O ZIP guarda protótipos com WebView2/MAUI; o EXE/APK posteriores são descritos como nativos. Não os misturamos.

Glossário essencial

Termo	Em palavras simples
Controller	Classe que recebe uma rota HTTP e chama o serviço.
Serviço	Classe que aplica a regra do sistema e usa o banco.
Modelo de domínio	Objeto com dados e comportamento do negócio, como Matricula.
DTO / Pedido	Classe que transporta dados de uma requisição, como DadosLogin.
Endpoint	Endereço da API, por exemplo /api/cursos.
JSON	Texto estruturado com nomes e valores enviado pela rede.
Cookie / Bearer	Duas formas de o cliente apresentar o token de sessão.
Hash / sal	Resultado irreversível para comparação de senha; o sal torna hashes diferentes.
Transação	Grupo de operações SQL que confirma junto ou desfaz junto.
Procedure / trigger	Rotina SQL chamada explicitamente / rotina que dispara com mudança na tabela.
WebView / Activity	Navegador interno / tela nativa de aplicativo Android.

Para organizar o estudo entre quatro pessoas

1º: visão geral, mapa e web. **2º:** API, segurança e fluxo do login. **3º:** curso, prova, certificado e banco. **4º:** entregas desktop/mobile, limites de verificação e ensaio da banca. Todos devem conseguir narrar o caminho interface → API → serviço → SQL → resposta.

Pontos para confirmar antes da apresentação: conseguir os fontes nativos do EXE/APK de 25/10, instalar/testar o APK em aparelho, abrir o EXE entregue e verificar a versão publicada e os dados atuais no Azure. O código web/API e os pacotes locais foram analisados; essas confirmações de execução não foram feitas neste guia.